Project Documentation — Personal Productivity Suite (m1)

Project: Personal Productivity Suite (CLI)
Folder: `m1/`
Language: Python (standard library only)
Run: `python main.py`

---

1. Project Overview & Objectives

Overview

This project is a **menu-driven CLI productivity suite** that provides three tools in one application:

- Calculator
- Notes Manager (persistent notes)
- Timer & Stopwatch

Objectives

- Build a modular Python CLI application using core programming concepts.
- Provide a simple, reliable toolset with clean input handling and persistent storage for notes.

---

2. Setup & Installation

Requirements

- Python **3.8+**
- No external packages (standard library only)

Steps

1. Clone the repository:
   ```
   git clone https://github.com/riteshraj-shetage/py-logs.git
   ```
2. Go to the module folder:
   ```
   cd py-logs/m1
   ```
3. Run:
   ```
   python main.py
   ```

---

3. Code Structure Explanation

```
m1/
  main.py
  requirements.txt
  modules/
    calculator.py
    notes_manager.py
    timer.py
  docs/
    user_guide.md
  data/
    notes.json   (created automatically when saving notes)
```

What each part does

- **main.py**: entry point; shows the main menu and routes to the selected tool.
- **modules/calculator.py**: arithmetic operations + session history (up to 20).
- **modules/notes_manager.py**: CRUD notes; saves/loads notes to/from JSON.
- **modules/timer.py**: countdown timer + stopwatch with laps.
- **data/notes.json**: persistent notes storage (auto-created).

---

4. User Manual (How to Use)

Main Menu

Run `python main.py`, then choose:

1. Calculator
2. Notes Manager
3. Timer & Stopwatch
4. Exit

Calculator

- Input format: `<number> <operator> <number>` (example: `25 / 4`)
- Operators: `+ - * /`
- Commands:
  - `h` = history
  - `b` = back
- Handles division by zero without crashing.

Notes Manager

Menu options:

- View all notes
- Add note (title + content)
- Search notes (keyword)

- Edit note (by ID)
- Delete note (by ID)

Notes are saved automatically in `data/notes.json`.

Timer & Stopwatch

- Countdown supports: `90` or `1:30` or `0:01:30`
- Stopwatch commands: `start`, `stop`, `lap`, `reset`, `back`

---

5. Module Explanations (Brief)

calculator.py

- `Calculator.calculate(a, op, b)`: performs calculation and records history.
- `run(calc)`: CLI loop; validates input and prints results.

notes_manager.py

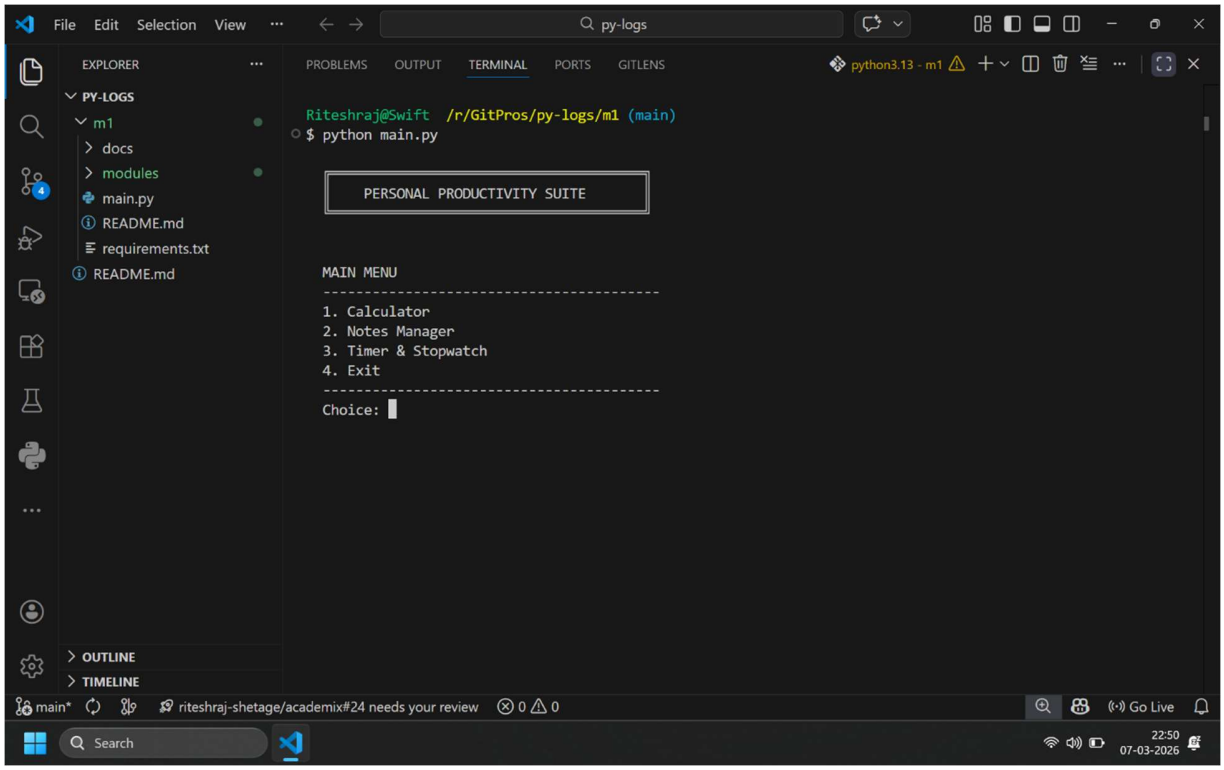
- `NotesManager`: loads/saves JSON and provides:
  - add, view, search, edit, delete
- `run(manager)`: CLI menu for note operations.

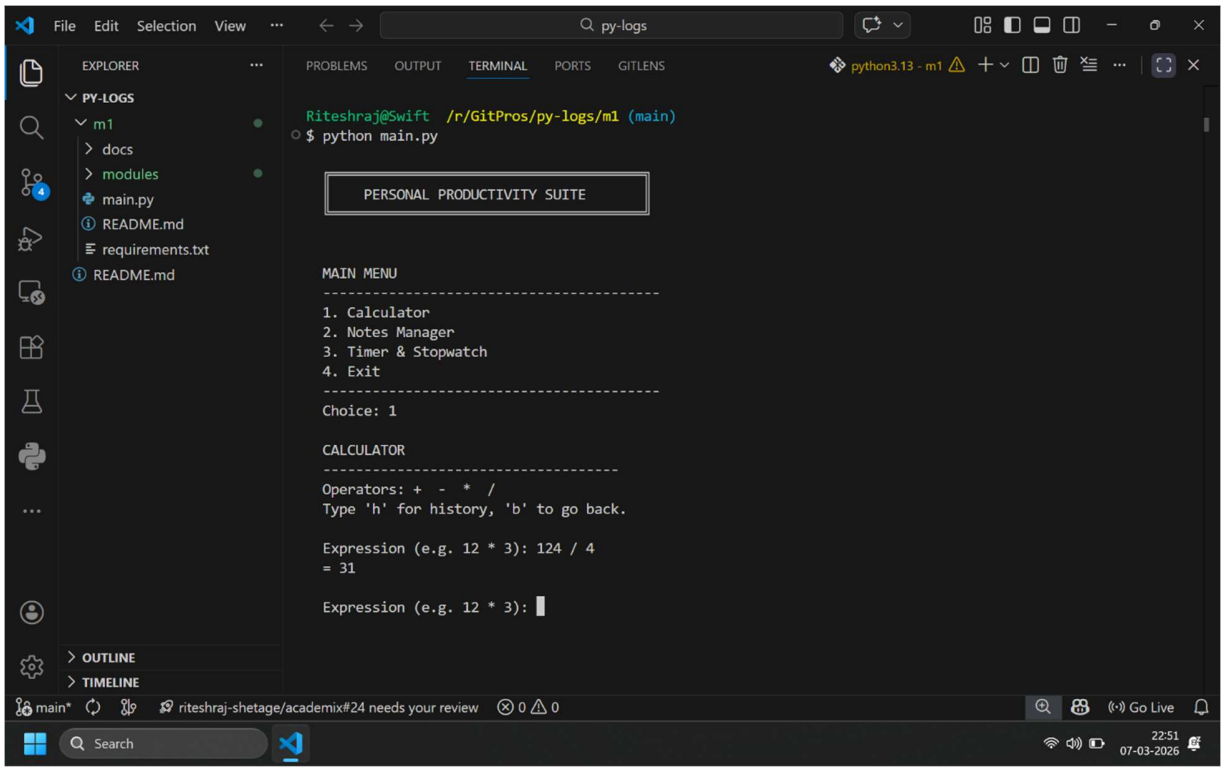
timer.py

- Countdown: parses duration and prints remaining time each second.
- `Stopwatch`: start/stop/lap/reset logic.
- `run()`: timer/stopwatch menu.

---

6. Screenshots of Working Application





```
4. Edit note
5. Delete note
6. Back to main menu

Choice: 2
Title: Project Updates
Content: Successfully completed project task for first month!
✓ Note #1 saved.

NOTES MANAGER
-----------------------------------
1. View all notes
2. Add note
3. Search notes
4. Edit note
5. Delete note
6. Back to main menu

Choice: b
! Invalid choice.

NOTES MANAGER
-----------------------------------
1. View all notes
2. Add note
3. Search notes
4. Edit note
5. Delete note
6. Back to main menu

Choice: 
```

```
Riteshraj@Swift  /r/GitPros/py-logs/m1 (main)
$ python main.py

    PERSONAL PRODUCTIVITY SUITE


MAIN MENU
----------------------------------------
1. Calculator
2. Notes Manager
3. Timer & Stopwatch
4. Exit
----------------------------------------
Choice: 3

TIMER & STOPWATCH
-----------------------------------
1. Countdown timer
2. Stopwatch
3. Back to main menu

Choice: 1
Duration (e.g. 90  or  1:30  or  0:01:30): 10

⏳ 00:00:05 remaining
```

7. Technical Requirements (How They're Met)

- **Modular programming:** features split across `modules/` and called from `main.py`.
- **OOP usage:** `Calculator`, `NotesManager`, `Stopwatch` classes.
- **File handling + persistence:** notes stored in `data/notes.json` using JSON read/write.
- **Error handling & validation:** handles invalid inputs, division by zero, invalid IDs, missing/invalid JSON file, and timer interruption (Ctrl+C).
- **No external dependencies:** runs using Python standard library only (`requirements.txt` confirms this).

---

8. Troubleshooting

- **`ModuleNotFoundError`**
  - Run from `py-logs/m1`:

    ```
    cd py-logs/m1
    python main.py
    ```

- **Notes not saving / `notes.json` missing**
  - Add at least one note; the file is created automatically.
  - Ensure the folder is writable.
- **Timer symbols look odd**
  - Terminal may not support Unicode; functionality is unaffected.
- **Invalid timer duration**
  - Use `90`, `1:30`, or `0:01:30`.